



28

Déploiement local & compose

US3.3 · C6

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Composer API + PostgreSQL avec healthchecks
- › Externaliser la config / secrets via .env
- › Écrire des smoke tests d'intégration
- › Distinguer smoke / intégration ; principes canary/rollback

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE docker compose up -d --wait → healthy · smoke test vert

1

Module 28 — les concepts clés

Théorie

■ Scénario atelier & enjeux

Ton service a besoin d'une base, d'un collecteur de métriques et d'un dashboard. Les lancer un par un est ingérable : compose démarre tout l'écosystème d'une seule commande.

Angle éthique / risque (C2) Des services qui se parlent, ce sont des données qui circulent. Cloisonner (réseaux, variables d'environnement) protège les accès.

■ Contexte & outils — pourquoi ce module

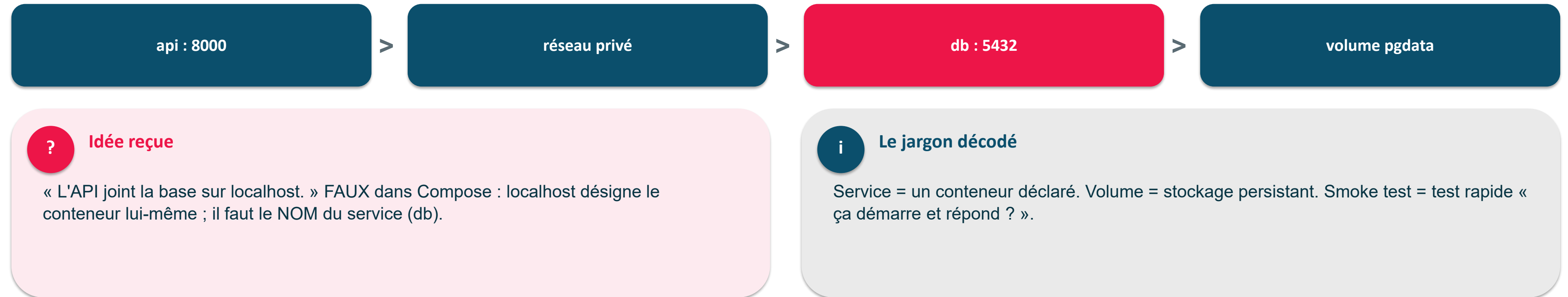
Contexte : une vraie appli, c'est plusieurs services. Pourquoi : on les orchestre ensemble (API + base + monitoring) en une commande.

- docker-compose : déclare les services et leur réseau
- PostgreSQL : la base de données du projet
- healthchecks : démarrage ordonné (la base prête avant l'API)
- .env : configuration et secrets HORS de l'image
- DNS interne : un service appelle l'autre par son nom (db:5432)

Objectif du module Toute la stack qui se lance avec un seul ``docker compose up``.

■ Plusieurs services qui se parlent : docker-compose

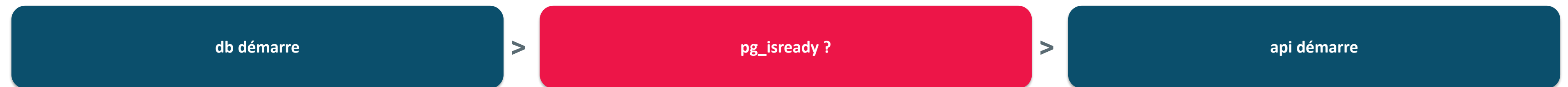
docker-compose décrit plusieurs services (API, base...) et leur réseau privé dans un seul fichier.



EN UNE PHRASE Dans le réseau compose, on joint un service par son nom (db:5432).

■ « Démarré » n'est pas « prêt » : les healthchecks

depends_on garantit qu'un service est lancé, pas qu'il est prêt — d'où le healthcheck.



! Attention — piège

Un bug intermittent au démarrage = quasi toujours une course : le service dépendant n'attend pas que l'autre soit PRÊT.

! À retenir

Config et secrets dans un .env non versionné ; on ne versionne que .env.example (les clés, sans les valeurs).

EN UNE PHRASE Healthcheck + condition: service_healthy = fin des bugs de démarrage.

■ Le docker-compose, en vrai (api + db + healthcheck)

Plusieurs services, un réseau privé : on joint par le nom (db), jamais localhost.

```
services:
  api:
    build: .
    depends_on: { db: { condition: service_healthy } }
    environment: { INDUSENSE_DB_URL: postgresql+psycopg://...@db:5432/indusense }
  db:
    image: postgres:16
    healthcheck: { test: ["CMD-SHELL","pg_isready -U indusense"] }
  prometheus: { image: prom/prometheus:v2.53.0 }
  grafana: { image: grafana/grafana:11.1.0 }
```

? Idée reçue

« L'API joint la base sur localhost. » FAUX : dans compose, localhost = le conteneur lui-même -> db:5432.

! Attention — piège

depends_on sans healthcheck = course au démarrage (plante par intermittence).

EN UNE PHRASE condition: service_healthy = on démarre l'API quand la base est PRÊTE, pas juste lancée.

■ À retenir en 3 points

- compose orchestre plusieurs services d'une seule commande.
- depends_on + healthcheck = démarrage ordonné.
- Les secrets vivent dans des variables d'environnement, jamais en dur.

2

Module 28 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — compose API + DB

```
services:
  api: { build: ., ports: ["8000:8000"],
        depends_on: { db: { condition: service_healthy } } }
  db: { image: postgres:16,
        healthcheck: { test: ["CMD-SHELL", "pg_isready -U indusense"] } }
```

✓ **Point de contrôle** docker compose ps → api + db HEALTHY.

TP — smoke tests

```
docker compose up -d --wait
uv run pytest tests/test_smoke_compose.py -q # test à créer
curl.exe -i http://localhost:8000/predict-tabular # 401 sans clé
curl.exe -H "X-API-Key: dev-key" -H "Content-Type: application/json" --data-binary "@payload.json" http://localhost:8000/predict-tabular # 200
```

DB jointe par db:5432 (nom de service), surtout pas localhost.

3

Module 28 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Autonomie tutorée — étendre la stack

Avancez à votre rythme. Le formateur reste disponible.

- Ajouter un service adminer (UI Postgres)
- OU brancher le smoke test dans la CI (module 24)

Posture en distanciel

Objectif : tout démarre d'une commande — `make up && make smoke`. Partagez votre docker compose ps en cas de service unhealthy.

■ Definition of Done & transition

- DoD : compose up healthy + smoke vert + .env propre

Transition → Module 29 La stack tourne ; on automatise le cycle de prédiction (Prefect).